PROGRAMACIÓN II · UNIDAD 1 · CLASE 3

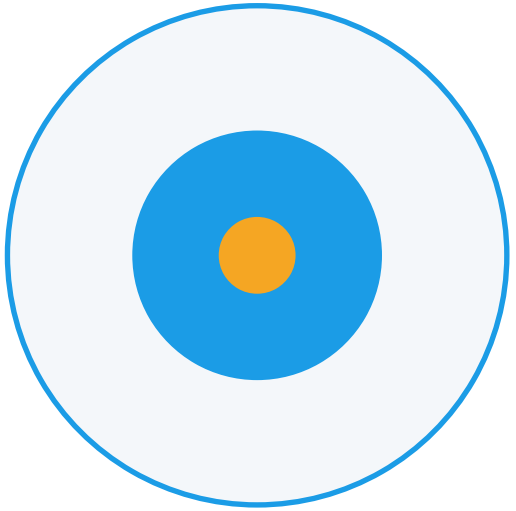
Construcción de objetos, sobrecarga y herencia

Bloque 2 · Programación Orientada a Objetos en C#

> *Crear e inicializar objetos con encapsulamiento, y dar el paso hacia la sobrecarga y la herencia.*

Objetivo de la clase

Crear e inicializar objetos con **encapsulamiento** e introducir la **sobrecarga** y la **herencia**.



Qué veremos hoy

1. Setters, getters y properties
2. Constructor, this y creación con new
3. Sobrecarga (overloading)
4. Herencia

1 Setters, getters y properties

Por convención, todo atributo es **privado** y se accede mediante **setters y getters**: funciones públicas de escritura y lectura.

```
// getters y setters

private string name;

public string GetName()
{
    return name;
}

public void SetName(string name)
{
    this.name = name;
}
```

```
// properties de C#

// forma abreviada (equivalente)
public string Name { get; set; }

// solo lectura
public int Id { get; }

// solo escritura / ambos:
// segun que accessor se declare
```

Se puede exponer un atributo solo para lectura, solo para escritura, o para ambos.

Apunte Unidad 1, pág. 10

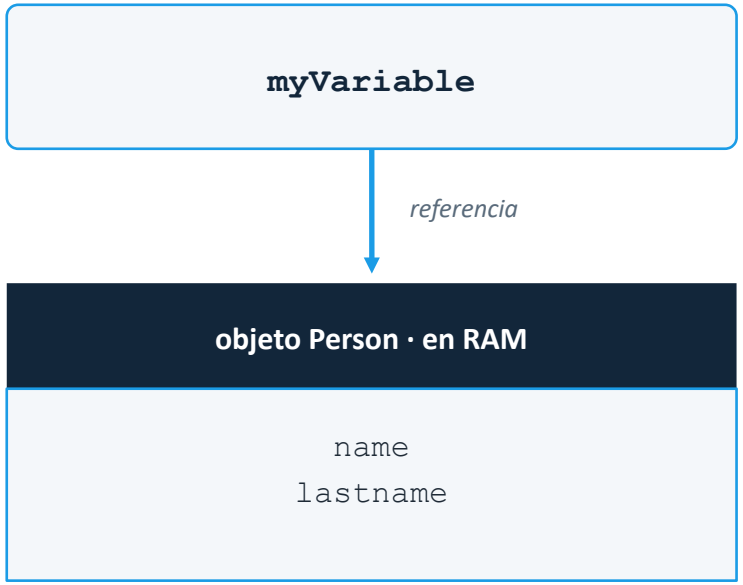
2 Creación de objetos: new y el accessor punto

new reserva espacio en la memoria RAM y devuelve la **referencia**, que se guarda en una variable del tipo de la clase. A sus miembros públicos se accede con el **punto (accessor)**.

Program.cs

```
Person myVariable = new Person();

myVariable.SetName("Mariano");
myVariable.SetLastname("Kaimakamian Carrau");
myVariable.SayHi();
```



Apunte Unidad 1, pág. 11-12 y 15-16

2 Constructor: por defecto vs. personalizado

Person.cs

```
public class Person
{
    private string name;
    private string lastname;

    public Person()                 // por defecto
    {
        this.name = "Juan";
        this.lastname = "Perez";
    }

    public Person(string name,      // personalizado
                  string lastname)
    {
        this.name = name;
        this.lastname = lastname;
    }
}
```

- Se ejecuta al crear la instancia; su firma no declara tipo de retorno.
- Es el lugar para las inicializaciones y condiciones obligatorias.
- Si no se define, se asume uno implícito (paréntesis vacíos).

Regla clave: si definís al menos un constructor, el vacío deja de estar disponible por defecto y debés agregarlo explícitamente.

Uso: `Person p = new Person("Al", "Goritmo");`

Apunte Unidad 1, pág. 12-14

2 La palabra reservada this

- Es una autorreferencia a la instancia (o a la clase, desde el diseño).
- Resuelve los homónimos: cuando un parámetro y un atributo tienen el mismo nombre, this identifica al miembro de la clase.
- Sin this, el compilador no sabría si la asignación va del parámetro al miembro o al revés.

constructor con homónimos

```
public Person(string name,
              string lastname)
{
    this.name = name;
    this.lastname = lastname;
}
```

this.name = miembro de la clase **name** = parámetro

Apunte Unidad 1, pág. 14-15

3 Sobrecarga (overloading)

Un mismo nombre de función con **distinta firma**: los parámetros deben diferir en tipo, cantidad y/o orden. El compilador elige la versión correcta según los argumentos.

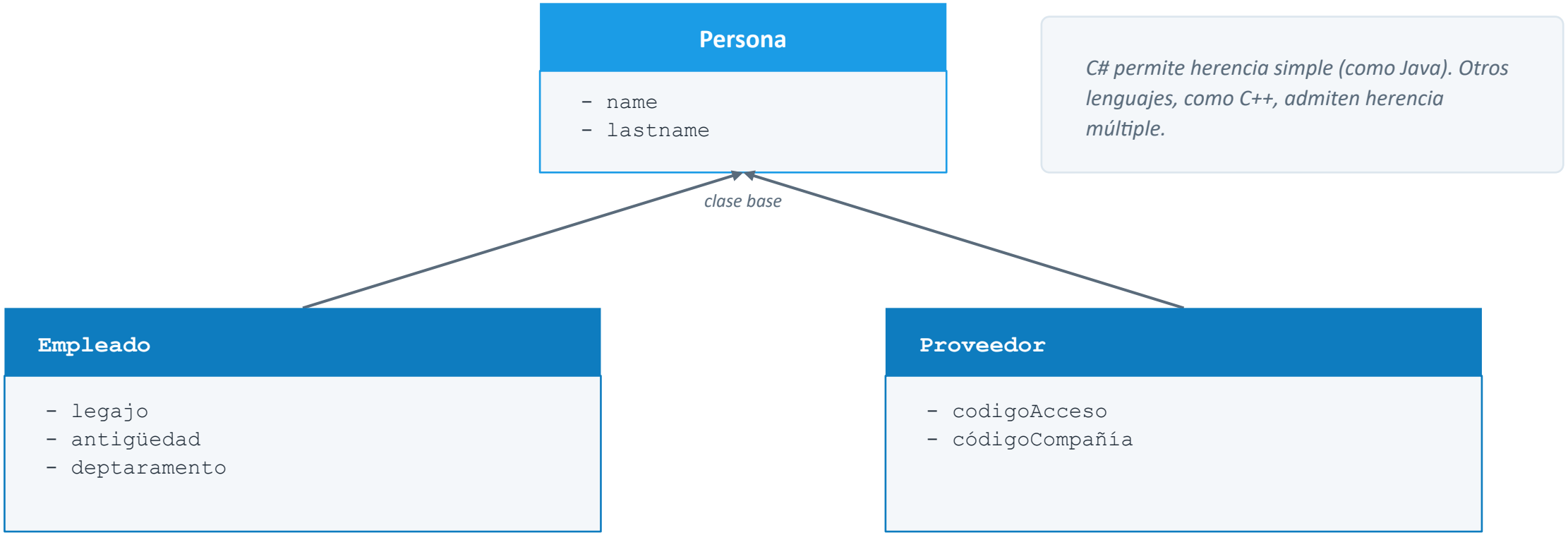
Visibilidad	Tipo	Nombre	Parámetros
public	string	`ConcatPersonInfo`	`(string name, string lastName)`
public	string	`ConcatPersonInfo`	`(string name, string middleName, string lastName)`
public	string	`ConcatPersonInfo`	`(string name, string lastName, int age)`

No compila: `getClientInfo(int account)` y `getClientInfo(int id)` tienen la misma firma: el compilador mira los tipos, no los nombres de los parámetros.

Apunte Unidad 1, pág. 16-18

4 Herencia

Crear una clase usando otra como base (relación de **sucesión**): la clase derivada usa los miembros **públicos y protegidos** del antecesor como si fueran suyos. Se declara con " **:** " seguido de la clase base.



C# permite herencia simple (como Java). Otros lenguajes, como C++, admiten herencia múltiple.

Apunte Unidad 1, pág. 19-20

EN SÍNTESIS

- Los atributos privados se exponen con getters/setters o con properties { get; set; }.
- new reserva memoria y devuelve la referencia; a los miembros se accede con el punto.
- El constructor inicializa la instancia; al definir uno, el vacío hay que agregarlo.
- this resuelve los homónimos entre parámetros y atributos.
- La sobrecarga: mismo nombre, distinta firma (tipo, cantidad u orden).
- La herencia (:) reutiliza los miembros públicos y protegidos de la clase base.

Próxima clase (práctica): múltiples constructores, objetos inmutables, sobrecarga de métodos y una primera jerarquía de clases.